

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Data Cleaning Challenge

NAME: S. Sharon Surya

REG NO: 192324246

COURSE Query Processing for Data

COURSE DSA0503

NAME: Science

CODE:

COURSE OUTCOME COVERED

CO3: Identify and clean inconsistencies in data by handling missing values, duplicates, outliers, formatting issues, and applying techniques like fuzzy matching and regex.

Assessment Tool Weightage: 35%

Total Marks: 20

Code of Conduct

I ___ S. Sharon Surya (192324246)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ___S. Sharon Surya _

Debugging

Criteria	Anomaly Detection (4 Marks)	Cleaning Methods (4 Marks)	Data Transformation (4 Marks)	Analytical Comparison (4 Marks)	Governance Discussion (4 Marks)
Marks					

**Total
Marks Awarded:**

Questions:

Question 1: E-Commerce Customer Transactions Dataset

An online retail company has collected customer transaction data from multiple sales channels. The management wants to identify the top-performing customer segments and products. However, the dataset contains several quality issues.

Customer	Customer Name	Gender	Age	Product Category	Quantity	Unit Price	Purchase Date	City
C101	Raj Kumar	Male	28	Electronics	2	15000 INR	10-01-2025	Chennai
C102	Priya S	Female	35	Fashion	3	1200 INR	11-01-2025	chennai
C103	Arun V	Male	150	Electronics	1	300 USD	12-01-2025	CHENNAI
C104	Kavya R	Female	0	Grocery	-2	800 INR	15-01-2025	Coimbatore
C105	Rahul P	Male	42	Fashion	5	2500 INR	18-01-2025	Madurai
C105	Rahul P	Male	42	Fashion	5	2500 INR	18-01-2025	Madurai
NULL	Sneha M	Female	30	Electronics	2	450 USD	05-02-2025	Trichy
C107	Deepak N	Male		Grocery	3	600 INR	07-02-2025	Tiruchirappalli
C108	Meena K	Female	27	Fashion	1	1800 INR	2025-13-01	Salem
C109	John D	Male	31	Electronics	4	250 EUR	15-02-2025	Chennai

Data Issues

- Missing Customer IDs
- Duplicate transactions
- Negative quantities
- Unit prices recorded in different currencies
- Age values such as 0, 150, and blanks
- Inconsistent city names (e.g., Chennai, chennai, CHENNAI)
- Incorrect purchase dates

Tasks

1. Identify and classify all data quality issues. (4 Marks)
2. Design a cleaning strategy for each issue. (4 Marks)
3. Perform data standardization and transformation. (4 Marks)
4. Analyze how cleaning affects total revenue calculations. (4 Marks)
5. Generate a report discussing business risks if the data were not cleaned (4 Marks)

CODE:

```
import pandas as pd
import numpy as np

# -----
# 1. CREATE THE DATASET
# -----

data = {
    "CustomerID": [
        "C101", "C102", "C103", "C104", "C105",
        "C105", None, "C107", "C108", "C109"
    ],
    "CustomerName": [
        "Raj Kumar", "Priya S", "Arun V", "Kavya R", "Rahul P",
        "Rahul P", "Sneha M", "Deepak N", "Meena K", "John D"
    ],
    "Gender": [
        "Male", "Female", "Male", "Female", "Male",
        "Male", "Female", "Male", "Female", "Male"
    ],
    "Age": [
        28, 35, 150, 0, 42, 42, 30, np.nan, 27, 31
    ],
    "ProductCategory": [
        "Electronics", "Fashion", "Electronics", "Grocery", "Fashion",
        "Fashion", "Electronics", "Grocery", "Fashion", "Electronics"
    ],
    "Quantity": [
        2, 3, 1, -2, 5, 5, 2, 3, 1, 4
    ],
    "UnitPrice": [
        "15000 INR", "1200 INR", "300 USD", "800 INR", "2500 INR",
        "2500 INR", "450 USD", "600 INR", "1800 INR", "250 EUR"
    ],
    "PurchaseDate": [
        "10-01-2025", "11-01-2025", "12-01-2025", "15-01-2025",
        "18-01-2025", "18-01-2025", "05-02-2025", "07-02-2025",
        "2025-13-01", "15-02-2025"
    ],
    "City": [
```

```

        "Chennai", "chennai", "CHENNAI", "Coimbatore", "Madurai",
        "Madurai", "Trichy", "Tiruchirappalli", "Salem", "Chennai"
    ]
}

```

```
df = pd.DataFrame(data)
```

```

print("===== ORIGINAL DATA =====")
print(df)

```

```

# -----
# 2. IDENTIFY DATA QUALITY ISSUES
# -----

```

```
print("\n===== DATA QUALITY ISSUES =====")
```

```

# Missing Customer IDs
print("\nMissing Customer IDs:")
print(df[df["CustomerID"].isna()])

```

```

# Duplicate transactions
print("\nDuplicate Transactions:")
print(df[df.duplicated(keep=False)])

```

```

# Negative quantities
print("\nNegative Quantities:")
print(df[df["Quantity"] < 0])

```

```

# Invalid ages
print("\nInvalid Ages:")
print(df[(df["Age"] <= 0) | (df["Age"] > 100) | (df["Age"].isna())])

```

```

# Invalid dates
print("\nInvalid Purchase Dates:")
converted_dates = pd.to_datetime(
    df["PurchaseDate"],
    errors="coerce",
    dayfirst=True
)
print(df[converted_dates.isna()])

```

```

# Different currencies
print("\nDifferent Currencies:")
print(df["UnitPrice"].str.extract(r"([A-Za-z]+)$")[0].unique())

```

```

# Inconsistent city names
print("\nCity Values:")
print(df["City"].unique())

```

```

# -----
# 3. CLEANING AND STANDARDIZATION
# -----

```

```
clean_df = df.copy()
```

```

# ----- Handle missing Customer IDs -----
# Generate a new ID for missing customer
clean_df.loc[clean_df["CustomerID"].isna(), "CustomerID"] = "C106"

# ----- Remove duplicate transactions -----
clean_df = clean_df.drop_duplicates()

# ----- Handle negative quantities -----
# Negative quantity is considered invalid.
# Replace it with NaN and later remove the transaction.
clean_df.loc[clean_df["Quantity"] < 0, "Quantity"] = np.nan

# ----- Clean Age -----
# Valid age assumed to be between 1 and 100.
clean_df.loc[
    (clean_df["Age"] <= 0) | (clean_df["Age"] > 100),
    "Age"
] = np.nan

# Fill missing age with median age
median_age = clean_df["Age"].median()
clean_df["Age"] = clean_df["Age"].fillna(median_age)

# -----
# 4. STANDARDIZE CURRENCY
# -----

# Extract numerical price
clean_df["Price"] = clean_df["UnitPrice"].str.extract(
    r"([\d.]+)"
)[0].astype(float)

# Extract currency
clean_df["Currency"] = clean_df["UnitPrice"].str.extract(
    r"([A-Za-z]+)$"
)[0].str.upper()

print("\nCurrencies found:")
print(clean_df["Currency"].value_counts())

# Currency conversion rates to INR
conversion_rates = {
    "INR": 1,
    "USD": 83,
    "EUR": 90
}

clean_df["Price_INR"] = clean_df.apply(
    lambda row: row["Price"] * conversion_rates.get(row["Currency"], np.nan),
    axis=1
)

```

```

# -----
# 5. STANDARDIZE PURCHASE DATE
# -----

clean_df["PurchaseDate"] = pd.to_datetime(
    clean_df["PurchaseDate"],
    errors="coerce",
    dayfirst=True
)

# Remove rows having invalid dates
clean_df = clean_df.dropna(subset=["PurchaseDate"])

# -----
# 6. STANDARDIZE CITY NAMES
# -----

city_mapping = {
    "chennai": "Chennai",
    "CHENNAI": "Chennai",
    "Chennai": "Chennai",
    "Trichy": "Tiruchirappalli",
    "Tiruchirappalli": "Tiruchirappalli"
}

clean_df["City"] = clean_df["City"].replace(city_mapping)

# -----
# 7. REMOVE INVALID QUANTITY TRANSACTIONS
# -----

clean_df = clean_df.dropna(subset=["Quantity"])

clean_df["Quantity"] = clean_df["Quantity"].astype(int)

# -----
# 8. CALCULATE REVENUE
# -----

clean_df["Revenue_INR"] = (
    clean_df["Quantity"] * clean_df["Price_INR"]
)

# -----
# 9. DISPLAY CLEANED DATA
# -----

print("\n===== CLEANED DATA =====")

print(
    clean_df[

```

```

        "CustomerID",
        "CustomerName",
        "Gender",
        "Age",
        "ProductCategory",
        "Quantity",
        "Price_INR",
        "PurchaseDate",
        "City",
        "Revenue_INR"
    ]
]
)

# -----
# 10. TOTAL REVENUE AFTER CLEANING
# -----

total_clean_revenue = clean_df["Revenue_INR"].sum()

print("\n===== REVENUE ANALYSIS =====")
print("Total Revenue After Cleaning:",
      total_clean_revenue, "INR")

# -----
# 11. REVENUE BEFORE CLEANING
# -----

# Create a temporary dataset for comparison
original_df = df.copy()

original_df["Price"] = original_df["UnitPrice"].str.extract(
    r"([\d.]+)"
)[0].astype(float)

original_df["Currency"] = original_df["UnitPrice"].str.extract(
    r"([A-Za-z]+)$"
)[0].str.upper()

original_df["Price_INR"] = original_df.apply(
    lambda row: row["Price"] *
    conversion_rates.get(row["Currency"], np.nan),
    axis=1
)

original_df["Revenue_INR"] = (
    original_df["Quantity"] * original_df["Price_INR"]
)

total_original_revenue = original_df["Revenue_INR"].sum()

print("Total Revenue Before Cleaning:",
      total_original_revenue, "INR")

```

```

# -----
# 12. REVENUE DIFFERENCE
# -----

revenue_difference = (
    total_original_revenue - total_clean_revenue
)

print("Revenue Difference:",
      revenue_difference, "INR")

# -----
# 13. TOP CUSTOMER SEGMENTS
# -----

print("\n===== TOP CUSTOMER SEGMENTS =====")

segment_revenue = clean_df.groupby(
    "Gender"
)["Revenue_INR"].sum().sort_values(ascending=False)

print(segment_revenue)

# -----
# 14. TOP PRODUCT CATEGORIES
# -----

print("\n===== TOP PRODUCT CATEGORIES =====")

category_revenue = clean_df.groupby(
    "ProductCategory"
)["Revenue_INR"].sum().sort_values(ascending=False)

print(category_revenue)

# -----
# 15. TOP CITIES
# -----

print("\n===== TOP CITIES =====")

city_revenue = clean_df.groupby(
    "City"
)["Revenue_INR"].sum().sort_values(ascending=False)

print(city_revenue)

# -----
# 16. BUSINESS RISK REPORT
# -----

print("\n===== BUSINESS RISK REPORT =====")

```

```
print("""
```

1. Missing Customer IDs:

Missing IDs make customer tracking difficult and can affect customer-level analysis and personalized marketing.

2. Duplicate Transactions:

Duplicate records artificially increase sales and revenue.

This can result in incorrect business decisions.

3. Negative Quantities:

Negative quantities may represent returns or data-entry errors.

If not handled correctly, revenue calculations become inaccurate.

4. Multiple Currencies:

INR, USD and EUR cannot be directly compared.

Without currency conversion, total revenue will be incorrect.

5. Invalid Ages:

Ages such as 0 and 150 are unrealistic and can distort

customer segmentation and demographic analysis.

6. Missing Ages:

Missing age values reduce the reliability of age-based

customer segmentation.

7. Inconsistent City Names:

Chennai, chennai and CHENNAI may be treated as different cities.

This produces incorrect city-wise sales reports.

8. Invalid Purchase Dates:

Incorrect dates can affect monthly, yearly and seasonal sales analysis.

9. Overall Business Risk:

If the dataset is not cleaned, management may identify the wrong top customers, products and cities. Revenue forecasts, marketing strategies and inventory decisions may therefore be inaccurate.

```
""")
```

OUTPUT:

```
IDLE Shell 3.12.10
File Edit Shell Debug Options Window Help
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/pavit/OneDrive. =====
===== ORIGINAL DATA =====
CustomerID CustomerName Gender ... UnitPrice PurchaseDate City
0 C101 Raj Kumar Male ... 15000 INR 10-01-2025 Chennai
1 C102 Priya S Female ... 1200 INR 11-01-2025 chennai
2 C103 Arun V Male ... 300 USD 12-01-2025 CHENNAI
3 C104 Kavya R Female ... 800 INR 15-01-2025 Coimbatore
4 C105 Rahul P Male ... 2500 INR 18-01-2025 Madurai
5 C105 Rahul P Male ... 2500 INR 18-01-2025 Madurai
6 NaN Sneha M Female ... 450 USD 05-02-2025 Trichy
7 C107 Deepak N Male ... 600 INR 07-02-2025 Tiruchirappalli
8 C108 Meena K Female ... 1800 INR 2025-13-01 Salem
9 C109 John D Male ... 250 EUR 15-02-2025 Chennai

[10 rows x 9 columns]

===== DATA QUALITY ISSUES =====

Missing Customer IDs:
CustomerID CustomerName Gender ... UnitPrice PurchaseDate City
6 NaN Sneha M Female ... 450 USD 05-02-2025 Trichy

[1 rows x 9 columns]

Duplicate Transactions:
CustomerID CustomerName Gender ... UnitPrice PurchaseDate City
4 C105 Rahul P Male ... 2500 INR 18-01-2025 Madurai
5 C105 Rahul P Male ... 2500 INR 18-01-2025 Madurai

[2 rows x 9 columns]

Negative Quantities:
CustomerID CustomerName Gender ... UnitPrice PurchaseDate City
3 C104 Kavya R Female ... 800 INR 15-01-2025 Coimbatore

[1 rows x 9 columns]

Invalid Ages:
CustomerID CustomerName Gender ... UnitPrice PurchaseDate City
2 C103 Arun V Male ... 300 USD 12-01-2025 CHENNAI
3 C104 Kavya R Female ... 800 INR 15-01-2025 Coimbatore
7 C107 Deepak N Male ... 600 INR 07-02-2025 Tiruchirappalli

[3 rows x 9 columns]
```

```
IDLE Shell 3.12.10
File Edit Shell Debug Options Window Help
<StringArray>
['INR', 'USD', 'EUR']
Length: 3, dtype: str

City Values:
<StringArray>
[ 'chennai', 'chennai', 'CHENNAI', 'Coimbatore',
  'Madurai', 'Trichy', 'Tiruchirappalli', 'Salem']
Length: 8, dtype: str

Currencies found:
Currency
INR 6
USD 2
EUR 1
Name: count, dtype: int64

===== CLEANED DATA =====
CustomerID CustomerName Gender ... PurchaseDate City Revenue_INR
0 C101 Raj Kumar Male ... 2025-01-10 Chennai 30000.0
1 C102 Priya S Female ... 2025-01-11 Chennai 3600.0
2 C103 Arun V Male ... 2025-01-12 Chennai 24900.0
4 C105 Rahul P Male ... 2025-01-18 Madurai 12500.0
6 C106 Sneha M Female ... 2025-02-05 Tiruchirappalli 74700.0
7 C107 Deepak N Male ... 2025-02-07 Tiruchirappalli 1800.0
9 C109 John D Male ... 2025-02-15 Chennai 90000.0

[7 rows x 10 columns]

===== REVENUE ANALYSIS =====
Total Revenue After Cleaning: 237500.0 INR
Total Revenue Before Cleaning: 250200.0 INR
Revenue Difference: 12700.0 INR

===== TOP CUSTOMER SEGMENTS =====
Gender
Male 159200.0
Female 78300.0
Name: Revenue_INR, dtype: float64

===== TOP PRODUCT CATEGORIES =====
ProductCategory
Electronics 219600.0
```

Question 2: Hospital Patient Records Dataset

A hospital wants to analyze patient admissions and treatment effectiveness. Data were entered manually by different departments, leading to inconsistencies.

PatientID	Name	Age	Gender	AdmissionDate	DischargeDate	Diagnosis	TreatmentCost	DoctorID
P1001	Ravi Kumar	45	M	10-01-2025	15-01-2025	Diabetes	25000	D101
P1002	Priya S	38	F	12-01-2025	11-01-2025	Hypertension	18000	D102
P1003	Arun V	125	M	13-01-2025	20-01-2025	diabetes	22000	D101
NULL	Kavya R	29	F	15-01-2025	17-01-2025	Diabetes	15000	D103
P1005	Rahul P	51	M	18-01-2025	25-01-2025	Asthma	NULL	D104
P1005	Rahul P	51	M	18-01-2025	25-01-2025	Asthma	NULL	D104
P1006	Sneha M	34	F	15-01-2025	18-01-2025	Hypertension	12000	D102
P1007	Deepak N		M	01-02-2025	05-02-2025	Asthma	10000	D104
P1008	Meena K	26	F	10-02-2025	15-02-2025	HYPERTENSION	13500	D102
P1009	John D	40	M	12-02-2025	18-02-2025	Diabetes	20000	D101

Data Issues

- ☐ Missing patient IDs
- ☐ Duplicate patient records

- ☐ Discharge dates before admission dates
- ☐ Null treatment costs
- ☐ Inconsistent diagnosis names
- ☐ Multiple formats of dates
- ☐ Unrealistic patient ages (>120)

Tasks

1. Detect all inconsistencies and anomalies. (4 Marks)
2. Develop rules for handling missing and invalid values. (4 Marks)
3. Create a process to standardize diagnosis names. (4 Marks)
4. Evaluate how cleaning changes average treatment cost and hospital stay duration. (4 Marks)
5. Recommend data governance measures to prevent future errors. (4 Marks)

CODE:

```
import pandas as pd
import numpy as np
```

```
# =====
# 1. CREATE THE DATASET
# =====
```

```
data = {
    "PatientID": [
        "P1001", "P1002", "P1003", None, "P1005",
        "P1005", "P1006", "P1007", "P1008", "P1009"
    ],
    "Name": [
        "Ravi Kumar", "Priya S", "Arun V", "Kavya R", "Rahul P",
        "Rahul P", "Sneha M", "Deepak N", "Meena K", "John D"
    ],
    "Age": [
        45, 38, 125, 29, 51, 51, 34, np.nan, 26, 40
    ],
    "Gender": [
        "M", "F", "M", "F", "M",
        "M", "F", "M", "F", "M"
    ],
    "AdmissionDate": [
        "10-01-2025", "12-01-2025", "13-01-2025", "15-01-2025",
        "18-01-2025", "18-01-2025", "15-01-2025", "01-02-2025",
        "10-02-2025", "12-02-2025"
    ],
    "DischargeDate": [
        "15-01-2025", "11-01-2025", "20-01-2025", "17-01-2025",
        "25-01-2025", "25-01-2025", "18-01-2025", "05-02-2025",
        "15-02-2025", "18-02-2025"
    ],
    "Diagnosis": [
        "Diabetes", "Hypertension", "diabetes", "Diabtes", "Asthma",
        "Asthma", "Hypertension", "Asthma", "HYPERTENSION", "Diabetes"
    ],
    "TreatmentCost": [
        25000, 18000, 22000, 15000, np.nan,
        np.nan, 12000, 10000, 13500, 20000
    ],
    "DoctorID": [
        "D101", "D102", "D101", "D103", "D104",
```

```

        "D104", "D102", "D104", "D102", "D101"
    ]
}

df = pd.DataFrame(data)

print("=" * 70)
print("ORIGINAL HOSPITAL PATIENT DATA")
print("=" * 70)
print(df)

# =====
# 2. TASK 1 - DETECT INCONSISTENCIES AND ANOMALIES
# =====

print("\n" + "=" * 70)
print("TASK 1: DATA QUALITY ISSUES")
print("=" * 70)

# Missing Patient IDs
print("\n1. Missing Patient IDs:")
print(df[df["PatientID"].isna()])

# Duplicate records
print("\n2. Duplicate Records:")
duplicates = df[df.duplicated(keep=False)]
print(duplicates)

# Invalid ages
print("\n3. Unrealistic Ages:")
invalid_age = df[
    (df["Age"] > 120) |
    (df["Age"] <= 0) |
    (df["Age"].isna())
]
print(invalid_age)

# Missing treatment cost
print("\n4. Missing Treatment Costs:")
print(df[df["TreatmentCost"].isna()])

# Convert dates temporarily for checking
admission_check = pd.to_datetime(
    df["AdmissionDate"],
    dayfirst=True,
    errors="coerce"
)

discharge_check = pd.to_datetime(
    df["DischargeDate"],

```

```

        dayfirst=True,
        errors="coerce"
    )

    # Discharge before admission
    print("\n5. Discharge Before Admission:")
    invalid_dates = df[
        discharge_check < admission_check
    ]
    print(invalid_dates)

    # Diagnosis inconsistencies
    print("\n6. Diagnosis Values Before Standardization:")
    print(df["Diagnosis"].unique())

# =====
# 3. TASK 2 - CLEANING RULES
# =====

clean_df = df.copy()

print("\n" + "=" * 70)
print("TASK 2: APPLYING CLEANING RULES")
print("=" * 70)

# -----
# RULE 1: Handle missing Patient ID
# -----

# Generate a new Patient ID for missing value
clean_df.loc[
    clean_df["PatientID"].isna(),
    "PatientID"
] = "P1004"

# -----
# RULE 2: Remove duplicate patient records
# -----

clean_df = clean_df.drop_duplicates()

# -----
# RULE 3: Handle unrealistic ages
# -----

# Ages must be between 1 and 120
clean_df.loc[
    (clean_df["Age"] <= 0) |
    (clean_df["Age"] > 120),
    "Age"
] = 1

```

```

] = np.nan

# Fill missing age with median age
median_age = clean_df["Age"].median()

clean_df["Age"] = clean_df["Age"].fillna(median_age)

# -----
# RULE 4: Standardize dates
# -----

clean_df["AdmissionDate"] = pd.to_datetime(
    clean_df["AdmissionDate"],
    dayfirst=True,
    errors="coerce"
)

clean_df["DischargeDate"] = pd.to_datetime(
    clean_df["DischargeDate"],
    dayfirst=True,
    errors="coerce"
)

# -----
# RULE 5: Handle discharge before admission
# -----

invalid_stay = (
    clean_df["DischargeDate"] <
    clean_df["AdmissionDate"]
)

# Mark invalid discharge dates as missing
clean_df.loc[
    invalid_stay,
    "DischargeDate"
] = pd.NaT

# -----
# RULE 6: Standardize diagnosis names
# -----

clean_df["Diagnosis"] = (
    clean_df["Diagnosis"]
    .str.strip()
    .str.lower()
)

diagnosis_mapping = {
    "diabetes": "Diabetes",
    "diabtes": "Diabetes",

```

```

        "hypertension": "Hypertension",
        "asthma": "Asthma"
    }

    clean_df["Diagnosis"] = clean_df[
        "Diagnosis"
    ].replace(diagnosis_mapping)

# -----
# RULE 7: Handle missing Treatment Cost
# -----

# Use median treatment cost
median_cost = clean_df["TreatmentCost"].median()

clean_df["TreatmentCost"] = clean_df[
    "TreatmentCost"
].fillna(median_cost)

# =====
# 4. CALCULATE HOSPITAL STAY
# =====

clean_df["HospitalStayDays"] = (
    clean_df["DischargeDate"] -
    clean_df["AdmissionDate"]
).dt.days

# Remove records where stay duration is invalid
clean_df = clean_df[
    clean_df["HospitalStayDays"].notna()
]

clean_df = clean_df[
    clean_df["HospitalStayDays"] >= 0
]

# =====
# 5. DISPLAY CLEANED DATA
# =====

print("\n" + "=" * 70)
print("CLEANED DATA")
print("=" * 70)

print(clean_df)

# =====
# 6. TASK 3 - DIAGNOSIS STANDARDIZATION RESULT
# =====

```

```

print("\n" + "=" * 70)
print("TASK 3: STANDARDIZED DIAGNOSIS")
print("=" * 70)

```

```

print(clean_df[
    ["PatientID", "Diagnosis"]
])

```

```

# =====
# 7. TASK 4 - AVERAGE TREATMENT COST
# =====

```

```

print("\n" + "=" * 70)
print("TASK 4: BEFORE VS AFTER CLEANING")
print("=" * 70)

```

```

# -----
# Average treatment cost BEFORE cleaning
# -----

```

```

average_cost_before = df["TreatmentCost"].mean()

```

```

# -----
# Average treatment cost AFTER cleaning
# -----

```

```

average_cost_after = clean_df["TreatmentCost"].mean()

```

```

print("\nAverage Treatment Cost BEFORE Cleaning:",
      round(average_cost_before, 2))

```

```

print("Average Treatment Cost AFTER Cleaning:",
      round(average_cost_after, 2))

```

```

# -----
# Average hospital stay BEFORE cleaning
# -----

```

```

original_dates_admission = pd.to_datetime(
    df["AdmissionDate"],
    dayfirst=True,
    errors="coerce"
)

```

```

original_dates_discharge = pd.to_datetime(
    df["DischargeDate"],
    dayfirst=True,
    errors="coerce"
)

```

```

original_stay = (
    original_dates_discharge -
    original_dates_admission
).dt.days

# Remove negative stays
original_stay = original_stay[
    original_stay >= 0
]

average_stay_before = original_stay.mean()

# -----
# Average hospital stay AFTER cleaning
# -----

average_stay_after = clean_df[
    "HospitalStayDays"
].mean()

print("\nAverage Hospital Stay BEFORE Cleaning:",
      round(average_stay_before, 2), "days")

print("Average Hospital Stay AFTER Cleaning:",
      round(average_stay_after, 2), "days")

# =====
# 8. CALCULATE CHANGES
# =====

cost_change = (
    average_cost_after -
    average_cost_before
)

stay_change = (
    average_stay_after -
    average_stay_before
)

print("\nTreatment Cost Change:",
      round(cost_change, 2))

print("Hospital Stay Change:",
      round(stay_change, 2), "days")

# =====
# 9. DIAGNOSIS-WISE ANALYSIS
# =====

```

```
print("\n" + "=" * 70)
print("DIAGNOSIS-WISE ANALYSIS")
print("=" * 70)
```

```
diagnosis_analysis = clean_df.groupby(
    "Diagnosis"
).agg(
    PatientCount=("PatientID", "count"),
    AverageTreatmentCost=("TreatmentCost", "mean"),
    AverageStayDays=("HospitalStayDays", "mean")
)

print(diagnosis_analysis.round(2))
```

```
# =====
# 10. TASK 5 - DATA GOVERNANCE RECOMMENDATIONS
# =====
```

```
print("\n" + "=" * 70)
print("TASK 5: DATA GOVERNANCE RECOMMENDATIONS")
print("=" * 70)
```

- ```
print("""
1. Use mandatory Patient IDs:
 PatientID should be automatically generated and should not
 be allowed to remain NULL.

2. Prevent duplicate records:
 Use unique constraints on PatientID and transaction/visit
 identifiers.

3. Validate patient age:
 Allow only realistic ages between 0 and 120.

4. Validate dates:
 DischargeDate must always be greater than or equal to
 AdmissionDate.

5. Standardize diagnosis:
 Use predefined diagnosis codes such as ICD codes instead
 of allowing free-text diagnosis entry.

6. Validate treatment cost:
 Treatment cost should not be NULL or negative.
 The system should require a valid numerical value.

7. Use dropdown menus:
 Gender, diagnosis and doctor information should be selected
 from predefined values instead of manual typing.

8. Date format standardization:
 Store dates in a standard format such as YYYY-MM-DD.

9. Automated data validation:
```

Apply validation rules when data is entered into the hospital management system.

10. Regular data quality audits:

Perform periodic checks for missing values, duplicates, invalid dates and inconsistent diagnosis names.

11. Role-based access:

Only authorized hospital staff should be allowed to modify patient records.

12. Maintain audit logs:

Record who created or modified a patient record and when the modification occurred.

""")

```
=====
11. FINAL SUMMARY
=====
```

```
print("\n" + "=" * 70)
print("FINAL SUMMARY")
print("=" * 70)
```

```
print("Original Number of Records :", len(df))
print("Cleaned Number of Records :", len(clean_df))
```

```
print("Average Treatment Cost Before :",
 round(average_cost_before, 2))
```

```
print("Average Treatment Cost After :",
 round(average_cost_after, 2))
```

```
print("Average Hospital Stay Before :",
 round(average_stay_before, 2), "days")
```

```
print("Average Hospital Stay After :",
 round(average_stay_after, 2), "days")
```

```
print("\nData cleaning completed successfully.")
```

## OUTPUT:

```
IDLE Shell 3.12.10
File Edit Shell Debug Options Window Help

===== RESTART: C: =====
ORIGINAL HOSPITAL PATIENT DATA
=====
PatientID Name Age ... Diagnosis TreatmentCost DoctorID
0 P1001 Ravi Kumar 45.0 ... Diabetes 25000.0 D101
1 P1002 Priya S 38.0 ... Hypertension 18000.0 D102
2 P1003 Arun V 125.0 ... diabetes 22000.0 D101
3 NaN Kavya R 29.0 ... Diabtes 15000.0 D103
4 P1005 Rahul P 51.0 ... Asthma NaN D104
5 P1005 Rahul P 51.0 ... Asthma NaN D104
6 P1006 Sneha M 34.0 ... Hypertension 12000.0 D102
7 P1007 Deepak N NaN ... Asthma 10000.0 D104
8 P1008 Meena K 26.0 ... HYPERTENSION 13500.0 D102
9 P1009 John D 40.0 ... Diabetes 20000.0 D101

[10 rows x 9 columns]

=====
TASK 1: DATA QUALITY ISSUES
=====

1. Missing Patient IDs:
PatientID Name Age ... Diagnosis TreatmentCost DoctorID
3 NaN Kavya R 29.0 ... Diabtes 15000.0 D103

[1 rows x 9 columns]

2. Duplicate Records:
PatientID Name Age ... Diagnosis TreatmentCost DoctorID
4 P1005 Rahul P 51.0 ... Asthma NaN D104
5 P1005 Rahul P 51.0 ... Asthma NaN D104

[2 rows x 9 columns]

3. Unrealistic Ages:
PatientID Name Age ... Diagnosis TreatmentCost DoctorID
2 P1003 Arun V 125.0 ... diabetes 22000.0 D101
7 P1007 Deepak N NaN ... Asthma 10000.0 D104

[2 rows x 9 columns]

4. Missing Treatment Costs:
PatientID Name Age ... Diagnosis TreatmentCost DoctorID
4 P1005 Rahul P 51.0 ... Asthma NaN D104
5 P1005 Rahul P 51.0 ... Asthma NaN D104
5 P1005 Rahul P 51.0 ... Asthma NaN D104

[2 rows x 9 columns]

5. Discharge Before Admission:
PatientID Name Age ... Diagnosis TreatmentCost DoctorID
1 P1002 Priya S 38.0 ... Hypertension 18000.0 D102

[1 rows x 9 columns]

6. Diagnosis Values Before Standardization:
<StringArray>
['Diabetes', 'Hypertension', 'diabetes', 'Diabtes', 'Asthma', 'HYPERTENSION']
Length: 6, dtype: str

=====
TASK 2: APPLYING CLEANING RULES
=====

CLEANED DATA
=====
PatientID Name Age ... TreatmentCost DoctorID HospitalStayDays
0 P1001 Ravi Kumar 45.0 ... 25000.0 D101 5.0
2 P1003 Arun V 38.0 ... 22000.0 D101 7.0
3 P1004 Kavya R 29.0 ... 15000.0 D103 2.0
4 P1005 Rahul P 51.0 ... 16500.0 D104 7.0
6 P1006 Sneha M 34.0 ... 12000.0 D102 3.0
7 P1007 Deepak N 38.0 ... 10000.0 D104 4.0
8 P1008 Meena K 26.0 ... 13500.0 D102 5.0
9 P1009 John D 40.0 ... 20000.0 D101 6.0

[8 rows x 10 columns]

=====
TASK 3: STANDARDIZED DIAGNOSIS
=====
PatientID Diagnosis
0 P1001 Diabetes
2 P1003 Diabetes
3 P1004 Diabetes
4 P1005 Asthma
```

### Question 3: Banking Loan Approval Dataset

A bank uses customer data to determine loan eligibility. The dataset contains errors that may affect risk assessment and approval decisions.

| CustomerID | AnnualIncome | CreditScore | LoanAmount | EmploymentStatus | Age | ExistingDebt | LoanStatus |
|------------|--------------|-------------|------------|------------------|-----|--------------|------------|
| L001       | 750000       | 720         | 300000     | Full Time        | 35  | 50000        | Approved   |
| L002       | 900K         | 680         | 400000     | FT               | 42  | 100000       | Approved   |
| L003       | 1200000      | NULL        | 600000     | fulltime         | 28  | 200000       | Approved   |
| L004       | 150000       | 450         | -200000    | Part Time        | 30  | 50000        | Rejected   |
| L005       | 5000000      | 790         | 1000000    | Self Employed    | 16  | 10000        | Approved   |
| L006       | 450000       | 350         | 250000     | Unemployed       | 40  | 300000       | Rejected   |
| L007       | 75000000     | 800         | 500000     | Full Time        | 45  | 100000       | Approved   |
| L008       | 650000       | 690         | 350000     | FT               | 33  | 70000        | Approved   |
| L008       | 650000       | 690         | 350000     | FT               | 33  | 70000        | Approved   |
| L010       | NULL         | 710         | 300000     | Full Time        | 37  | 80000        | Approved   |

#### Data Issues

- ☐ Missing credit scores
- ☐ Income values recorded in different units
- ☐ Negative loan amounts
- ☐ Duplicate customer IDs
- ☐ Age values below 18
- ☐ Employment status entered differently (FT, Full Time, fulltime)

☐ Extreme outliers in income

### Tasks

1. Identify critical issues that could impact loan decisions. (4 Marks)
2. Propose cleaning techniques for missing and inconsistent data. (4 Marks)
3. Detect and justify treatment of outliers. (4 Marks)
4. Analyze the effect of cleaning on loan approval rates. (4 Marks)
5. Discuss ethical implications of incorrect data cleaning decisions. (4 Marks)

### CODE:

```
import pandas as pd
import numpy as np

=====
1. CREATE THE DATASET
=====

data = {
 "CustomerID": [
 "L001", "L002", "L003", "L004", "L005",
 "L006", "L007", "L008", "L008", "L010"
],
 "AnnualIncome": [
 "750000", "900K", "1200000", "150000",
 "5000000", "450000", "75000000", "650000",
 "650000", None
],
 "CreditScore": [
 720, 680, np.nan, 450, 790,
 350, 800, 690, 690, 710
],
 "LoanAmount": [
 300000, 400000, 600000, -200000, 1000000,
 250000, 500000, 350000, 350000, 300000
],
 "EmploymentStatus": [
 "Full Time", "FT", "fulltime", "Part Time",
 "Self Employed", "Unemployed", "Full Time",
 "FT", "FT", "Full Time"
],
 "Age": [
 35, 42, 28, 30, 16,
 40, 45, 33, 33, 37
],
 "ExistingDebt": [
 50000, 100000, 200000, 50000, 10000,
 300000, 100000, 70000, 70000, 80000
],
 "LoanStatus": [
 "Approved", "Approved", "Approved", "Rejected",
 "Approved", "Rejected", "Approved", "Approved",
 "Approved", "Approved"
]
}

df = pd.DataFrame(data)

print("=" * 70)
print("ORIGINAL BANKING LOAN DATA")
print("=" * 70)
```

```

print(df)

=====
TASK 1 - IDENTIFY CRITICAL ISSUES
=====

print("\n" + "=" * 70)
print("TASK 1: CRITICAL DATA ISSUES")
print("=" * 70)

Missing credit scores
print("\n1. Missing Credit Scores:")
print(df[df["CreditScore"].isna()])

Missing income
print("\n2. Missing Annual Income:")
print(df[df["AnnualIncome"].isna()])

Duplicate customer IDs
print("\n3. Duplicate Customer IDs:")
print(df[df.duplicated("CustomerID", keep=False)])

Negative loan amounts
print("\n4. Negative Loan Amounts:")
print(df[df["LoanAmount"] < 0])

Age below 18
print("\n5. Age Below 18:")
print(df[df["Age"] < 18])

Employment status inconsistencies
print("\n6. Employment Status Values:")
print(df["EmploymentStatus"].unique())

Income values
print("\n7. Annual Income Values:")
print(df["AnnualIncome"])

=====
TASK 2 - CLEANING
=====

clean_df = df.copy()

print("\n" + "=" * 70)
print("TASK 2: DATA CLEANING")
print("=" * 70)

1. Standardize Annual Income

```

```

Convert values such as 900K to 900000
def convert_income(value):

 if pd.isna(value):
 return np.nan

 value = str(value).strip().upper()

 if value.endswith("K"):
 return float(value[:-1]) * 1000

 if value.endswith("M"):
 return float(value[:-1]) * 1000000

 return float(value)

clean_df["AnnualIncome"] = clean_df[
 "AnnualIncome"
].apply(convert_income)

2. Handle missing income

median_income = clean_df["AnnualIncome"].median()

clean_df["AnnualIncome"] = clean_df[
 "AnnualIncome"
].fillna(median_income)

3. Handle missing Credit Score

median_credit = clean_df["CreditScore"].median()

clean_df["CreditScore"] = clean_df[
 "CreditScore"
].fillna(median_credit)

4. Remove duplicate customer records

clean_df = clean_df.drop_duplicates(
 subset="CustomerID",
 keep="first"
)

5. Handle negative Loan Amount

Negative loan amount is invalid.

```

```

Replace it with missing value.
clean_df.loc[
 clean_df["LoanAmount"] < 0,
 "LoanAmount"
] = np.nan

Fill invalid loan amount using median
median_loan = clean_df["LoanAmount"].median()

clean_df["LoanAmount"] = clean_df[
 "LoanAmount"
].fillna(median_loan)

6. Handle Age below 18

Age below 18 is invalid for a normal loan applicant.
clean_df.loc[
 clean_df["Age"] < 18,
 "Age"
] = np.nan

Fill invalid age using median age
median_age = clean_df["Age"].median()

clean_df["Age"] = clean_df[
 "Age"
].fillna(median_age)

7. Standardize Employment Status

employment_mapping = {
 "FT": "Full Time",
 "fulltime": "Full Time",
 "Full Time": "Full Time",
 "Part Time": "Part Time",
 "Self Employed": "Self Employed",
 "Unemployed": "Unemployed"
}

clean_df["EmploymentStatus"] = (
 clean_df["EmploymentStatus"]
 .str.strip()
 .replace(employment_mapping)
)

=====
TASK 3 - OUTLIER DETECTION
=====

print("\n" + "=" * 70)
print("TASK 3: OUTLIER DETECTION")
print("=" * 70)

```

```

IQR method for Annual Income

Q1 = clean_df["AnnualIncome"].quantile(0.25)
Q3 = clean_df["AnnualIncome"].quantile(0.75)

IQR = Q3 - Q1

lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR

print("\nQ1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)

print("\nLower Outlier Limit:", lower_limit)
print("Upper Outlier Limit:", upper_limit)

outliers = clean_df[
 (clean_df["AnnualIncome"] < lower_limit) |
 (clean_df["AnnualIncome"] > upper_limit)
]

print("\nDetected Income Outliers:")
print(outliers[
 ["CustomerID", "AnnualIncome"]
])

Treat income outlier

Instead of deleting the customer, cap the income
using the IQR upper limit.
clean_df["AnnualIncome_Cleaned"] = clean_df[
 "AnnualIncome"
].clip(
 lower=lower_limit,
 upper=upper_limit
)

print("\nIncome After Outlier Treatment:")
print(clean_df[
 ["CustomerID", "AnnualIncome", "AnnualIncome_Cleaned"]
])

=====
TASK 4 - EFFECT ON LOAN APPROVAL RATE
=====

print("\n" + "=" * 70)
print("TASK 4: LOAN APPROVAL RATE ANALYSIS")
print("=" * 70)

```

```

Approval rate BEFORE cleaning

approval_before = (
 df["LoanStatus"]
 .eq("Approved")
 .mean()
) * 100

Approval rate AFTER cleaning

approval_after = (
 clean_df["LoanStatus"]
 .eq("Approved")
 .mean()
) * 100

print("\nLoan Approval Rate BEFORE Cleaning:",
 round(approval_before, 2), "%")

print("Loan Approval Rate AFTER Cleaning:",
 round(approval_after, 2), "%")

Change in approval rate

approval_change = approval_after - approval_before

print("Change in Approval Rate:",
 round(approval_change, 2), "%")

=====
APPROVAL ANALYSIS BY EMPLOYMENT STATUS
=====

print("\n" + "=" * 70)
print("APPROVAL RATE BY EMPLOYMENT STATUS")
print("=" * 70)

employment_approval = clean_df.groupby(
 "EmploymentStatus"
)["LoanStatus"].apply(
 lambda x: (x == "Approved").mean() * 100
)

print(employment_approval.round(2))

=====
CREDIT SCORE ANALYSIS
=====

```

```
print("\n" + "=" * 70)
print("CREDIT SCORE ANALYSIS")
print("=" * 70)
```

```
print(
 clean_df[
 ["CustomerID", "CreditScore", "LoanStatus"]
]
)
```

```
=====
INCOME ANALYSIS
=====
```

```
print("\n" + "=" * 70)
print("INCOME ANALYSIS")
print("=" * 70)
```

```
print(
 clean_df[
 [
 "CustomerID",
 "AnnualIncome",
 "AnnualIncome_Cleaned",
 "LoanStatus"
]
]
)
```

```
=====
TASK 5 - ETHICAL IMPLICATIONS
=====
```

```
print("\n" + "=" * 70)
print("TASK 5: ETHICAL IMPLICATIONS")
print("=" * 70)
```

- ```
print("""
1. Unfair Loan Decisions:
    Incorrectly changing credit scores or income values can cause
    eligible customers to be rejected or risky customers to be
    approved.

2. Discrimination:
    Data cleaning should not introduce bias against customers based
    on employment status, age, gender or other protected attributes.

3. Transparency:
    The bank should document why missing values, duplicates and
    outliers were changed or removed.

4. Outlier Treatment:
    A very high income should not automatically be treated as an
    error. It may represent a genuine high-income customer.
    Therefore, outliers should be investigated before removal.

5. Privacy:
```

Financial and personal information must be securely stored and accessed only by authorized personnel.

6. Accuracy:

Credit scores and financial information are highly important for loan decisions. Incorrect imputation can directly affect a customer's financial opportunities.

7. Human Oversight:

Automated loan decisions should have monitoring and suitable human review mechanisms, especially for unusual cases.

8. Auditability:

Every modification to customer data should be recorded so that decisions can be reviewed later.

""")

```
# =====  
# FINAL SUMMARY  
# =====
```

```
print("\n" + "=" * 70)  
print("FINAL SUMMARY")  
print("=" * 70)  
  
print("Original Number of Records:",  
      len(df))  
  
print("Cleaned Number of Records:",  
      len(clean_df))  
  
print("Missing Credit Scores:",  
      df["CreditScore"].isna().sum())  
  
print("Duplicate Customer IDs:",  
      df["CustomerID"].duplicated().sum())  
  
print("Negative Loan Amounts:",  
      (df["LoanAmount"] < 0).sum())  
  
print("Applicants Below 18:",  
      (df["Age"] < 18).sum())  
  
print("Income Outliers Detected:",  
      len(outliers))  
  
print("Approval Rate Before Cleaning:",  
      round(approval_before, 2), "%")  
  
print("Approval Rate After Cleaning:",  
      round(approval_after, 2), "%")  
  
print("\nData cleaning and analysis completed successfully.")
```

OUTPUT:

```
IDLE Shell 3.12.10
File Edit Shell Debug Options Window Help

=====
ORIGINAL BANKING LOAN DATA
=====
CustomerID AnnualIncome CreditScore ... Age ExistingDebt LoanStatus
0 L001 750000 720.0 ... 35 50000 Approved
1 L002 900K 680.0 ... 42 100000 Approved
2 L003 1200000 NaN ... 28 200000 Approved
3 L004 150000 450.0 ... 30 50000 Rejected
4 L005 5000000 790.0 ... 16 10000 Approved
5 L006 450000 350.0 ... 40 300000 Rejected
6 L007 75000000 800.0 ... 45 100000 Approved
7 L008 650000 690.0 ... 33 70000 Approved
8 L008 650000 690.0 ... 33 70000 Approved
9 L010 NaN 710.0 ... 37 80000 Approved

[10 rows x 8 columns]

=====
TASK 1: CRITICAL DATA ISSUES
=====

1. Missing Credit Scores:
CustomerID AnnualIncome CreditScore ... Age ExistingDebt LoanStatus
2 L003 1200000 NaN ... 28 200000 Approved

[1 rows x 8 columns]

2. Missing Annual Income:
CustomerID AnnualIncome CreditScore ... Age ExistingDebt LoanStatus
9 L010 NaN 710.0 ... 37 80000 Approved

[1 rows x 8 columns]

3. Duplicate Customer IDs:
CustomerID AnnualIncome CreditScore ... Age ExistingDebt LoanStatus
7 L008 650000 690.0 ... 33 70000 Approved
8 L008 650000 690.0 ... 33 70000 Approved

[2 rows x 8 columns]

4. Negative Loan Amounts:
CustomerID AnnualIncome CreditScore ... Age ExistingDebt LoanStatus
3 L004 150000 450.0 ... 30 50000 Rejected

[1 rows x 8 columns]

=====
5. Age Below 18:
CustomerID AnnualIncome CreditScore ... Age ExistingDebt LoanStatus
4 L005 5000000 790.0 ... 16 10000 Approved

[1 rows x 8 columns]

6. Employment Status Values:
<StringArray>
['Full Time', 'FT', 'fulltime', 'Part Time', 'Self Employed', 'Unemployed']
Length: 6, dtype: str

7. Annual Income Values:
0 750000
1 900K
2 1200000
3 150000
4 5000000
5 450000
6 75000000
7 650000
8 650000
9 NaN
Name: AnnualIncome, dtype: str

=====
TASK 2: DATA CLEANING
=====

=====
TASK 3: OUTLIER DETECTION
=====

Q1: 650000.0
Q3: 1200000.0
IQR: 550000.0

Lower Outlier Limit: -175000.0
Upper Outlier Limit: 2025000.0

Detected Income Outliers:
CustomerID AnnualIncome
4 L005 5000000.0
6 L007 75000000.0

Income After Outlier Treatment:
CustomerID AnnualIncome AnnualIncome_Cleaned
0 L001 750000.0 750000.0
```

Question 4: Smart City Traffic Monitoring Dataset

A smart city project collects traffic sensor data to optimize traffic signals. Sensor malfunctions and transmission errors have affected data quality.

SensorID	Timestamp	VehicleCount	AverageSpeed	TrafficSignalStatus	RoadName	WeatherCondition
S101	01-01-2025 08:00	120	45	Green	Anna Salai	Sunny
S102	01-01-2025 08:05	135	52	Red	anna salai	Sunny
S103	01-01-2025 08:10	-50	48	Green	ANNA SALAI	Cloudy
S104	01-01-2025 08:15	140	350	Yellow	GST Road	Rainy
S105	NULL	110	42	Green	GST ROAD	Sunny
S106	01-01-2025 08:25	130	46	Red	OMR	NULL
S106	01-01-2025 08:25	130	46	Red	OMR	NULL
S107	01-01-2025 08:30	150	55	Green	Old Mahabalipuram Road	Sunny
S10A	01-01-2025 08:35	145	53	Green	OMR	Cloudy
S109	01-01-2025 08:40	138	50	Green	GST Road	Rainy

Data Issues

- ☐ Missing timestamps
- ☐ Duplicate sensor readings
- ☐ Vehicle counts recorded as negative numbers
- ☐ Average speed exceeding 300 km/h
- ☐ Missing weather information
- ☐ Inconsistent road naming conventions
- ☐ Sensor IDs with formatting errors

Tasks

1. Assess the quality of the traffic dataset. (4 Marks)
2. Recommend cleaning techniques for time-series data. (4 Marks)
3. Design rules to identify faulty sensor readings. (4 Marks)
4. Evaluate how cleaning affects congestion analysis. (4 Marks)
5. Suggest automated monitoring techniques for future data collection. (4 Marks)

CODE:

```
import pandas as pd
import numpy as np
```

```
# =====
# QUESTION 4: SMART CITY TRAFFIC MONITORING DATASET
# =====

# =====
# 1. CREATE THE DATASET
# =====

data = {
    "SensorID": [
        "S101", "S102", "S103", "S104", "S105",
        "S106", "S106", "S107", "S10A", "S109"
    ],

    "Timestamp": [
        "01-01-2025 08:00",
        "01-01-2025 08:05",
        "01-01-2025 08:10",
        "01-01-2025 08:15",
        None,
        "01-01-2025 08:25",
        "01-01-2025 08:25",
        "01-01-2025 08:30",
        "01-01-2025 08:35",
        "01-01-2025 08:40"
    ],

    "VehicleCount": [
        120, 135, -50, 140, 110,
        130, 130, 150, 145, 138
    ],

    "AverageSpeed": [
        45, 52, 48, 350, 42,
        46, 46, 55, 53, 50
    ],

    "TrafficSignalStatus": [
        "Green", "Red", "Green", "Yellow", "Green",
        "Red", "Red", "Green", "Green", "Green"
    ]
}
```

```

    ],

    "RoadName": [
        "Anna Salai",
        "anna salai",
        "ANNA SALAI",
        "GST Road",
        "GST ROAD",
        "OMR",
        "OMR",
        "Old Mahabalipuram Road",
        "OMR",
        "GST Road"
    ],

    "WeatherCondition": [
        "Sunny", "Sunny", "Cloudy", "Rainy", "Sunny",
        None, None, "Sunny", "Cloudy", "Rainy"
    ]
}

df = pd.DataFrame(data)

print("=" * 75)
print("ORIGINAL SMART CITY TRAFFIC DATA")
print("=" * 75)

print(df)

# =====
# TASK 1: ASSESS DATA QUALITY
# =====

print("\n" + "=" * 75)
print("TASK 1: DATA QUALITY ASSESSMENT")
print("=" * 75)

# -----
# 1. Missing timestamps
# -----

print("\n1. Missing Timestamps:")

print(
    df[df["Timestamp"].isna()]
)

# -----
# 2. Duplicate sensor readings
# -----

print("\n2. Duplicate Sensor Readings:")

duplicates = df[
    df.duplicated(
        subset=["SensorID", "Timestamp"],
        keep=False
    )
]

```

```

    )
]

print(duplicates)

# -----
# 3. Negative vehicle counts
# -----

print("\n3. Negative Vehicle Counts:")

print(
    df[df["VehicleCount"] < 0]
)

# -----
# 4. Unrealistic speed
# -----

print("\n4. Unrealistic Average Speed:")

print(
    df[df["AverageSpeed"] > 300]
)

# -----
# 5. Missing weather
# -----

print("\n5. Missing Weather Information:")

print(
    df[df["WeatherCondition"].isna()]
)

# -----
# 6. Road name inconsistencies
# -----

print("\n6. Road Names Before Standardization:")

print(df["RoadName"].unique())

# -----
# 7. Sensor ID formatting errors
# -----

print("\n7. Sensor IDs:")

print(df["SensorID"].unique())

print("\nSensor IDs with formatting errors:")

invalid_sensor = df[
    ~df["SensorID"].str.match(r"^[S\d]{3}$")

```

```

]

print(invalid_sensor)

# =====
# TASK 2: CLEANING TIME-SERIES DATA
# =====

clean_df = df.copy()

print("\n" + "=" * 75)
print("TASK 2: TIME-SERIES DATA CLEANING")
print("=" * 75)

# -----
# 1. Convert timestamp to datetime
# -----

clean_df["Timestamp"] = pd.to_datetime(
    clean_df["Timestamp"],
    format="%d-%m-%Y %H:%M",
    errors="coerce"
)

# -----
# 2. Handle missing timestamp
# -----

# Sort data based on timestamp
clean_df = clean_df.sort_values(
    "Timestamp"
)

# Interpolate missing timestamp based on surrounding values
clean_df["Timestamp"] = (
    clean_df["Timestamp"]
    .interpolate()
)

# -----
# 3. Remove duplicate sensor readings
# -----

clean_df = clean_df.drop_duplicates(
    subset=["SensorID", "Timestamp"],
    keep="first"
)

# -----
# 4. Handle negative vehicle counts
# -----

# Negative vehicle count is impossible.
# Replace with NaN.

```

```

clean_df.loc[
    clean_df["VehicleCount"] < 0,
    "VehicleCount"
] = np.nan

# Fill missing vehicle count using interpolation
clean_df["VehicleCount"] = (
    clean_df["VehicleCount"]
    .interpolate()
)

# -----
# 5. Handle unrealistic speed
# -----

# Maximum reasonable city speed assumed as 120 km/h.
clean_df.loc[
    (clean_df["AverageSpeed"] < 0) |
    (clean_df["AverageSpeed"] > 120),
    "AverageSpeed"
] = np.nan

# Interpolate speed
clean_df["AverageSpeed"] = (
    clean_df["AverageSpeed"]
    .interpolate()
)

# -----
# 6. Standardize Road Names
# -----

road_mapping = {
    "anna salai": "Anna Salai",
    "ANNA SALAI": "Anna Salai",
    "Anna Salai": "Anna Salai",

    "GST Road": "GST Road",
    "GST ROAD": "GST Road",

    "OMR": "Old Mahabalipuram Road",
    "Old Mahabalipuram Road":
        "Old Mahabalipuram Road"
}

clean_df["RoadName"] = (
    clean_df["RoadName"]
    .str.strip()
    .replace(road_mapping)
)

# -----
# 7. Handle missing weather
# -----

```

```

# Use forward fill followed by backward fill
clean_df["WeatherCondition"] = (
    clean_df["WeatherCondition"]
    .ffill()
    .bfill()
)

# =====
# TASK 3: SENSOR FAULT DETECTION RULES
# =====

print("\n" + "=" * 75)
print("TASK 3: FAULTY SENSOR READING DETECTION")
print("=" * 75)

# -----
# Rule 1: Vehicle count must not be negative
# -----

fault_vehicle = clean_df[
    clean_df["VehicleCount"] < 0
]

print("\nFaulty Vehicle Counts:")
print(fault_vehicle)

# -----
# Rule 2: Speed must be within reasonable range
# -----

fault_speed = clean_df[
    (clean_df["AverageSpeed"] < 0) |
    (clean_df["AverageSpeed"] > 120)
]

print("\nFaulty Speed Readings:")
print(fault_speed)

# -----
# Rule 3: Sensor ID format
# -----

fault_sensor_id = clean_df[
    ~clean_df["SensorID"].str.match(r"^[S\d]{3}$")
]

print("\nFaulty Sensor IDs:")
print(fault_sensor_id)

# -----
# Rule 4: Duplicate readings
# -----

fault_duplicates = clean_df[
    clean_df.duplicated(

```

```

        subset=["SensorID", "Timestamp"],
        keep=False
    )
]

print("\nDuplicate Readings:")
print(fault_duplicates)

# =====
# CONGESTION CLASSIFICATION
# =====

print("\n" + "=" * 75)
print("CONGESTION ANALYSIS")
print("=" * 75)

# -----
# Define congestion rules
# -----

def classify_congestion(row):

    if (
        row["AverageSpeed"] < 30
        or row["VehicleCount"] > 140
    ):
        return "High"

    elif (
        row["AverageSpeed"] < 45
        or row["VehicleCount"] > 120
    ):
        return "Medium"

    else:
        return "Low"

clean_df["CongestionLevel"] = clean_df.apply(
    classify_congestion,
    axis=1
)

print("\nCongestion Level After Cleaning:")

print(
    clean_df[
        [
            "SensorID",
            "Timestamp",
            "VehicleCount",
            "AverageSpeed",
            "RoadName",
            "CongestionLevel"
        ]
    ]
)

```

```

# =====
# TASK 4: COMPARE CONGESTION BEFORE AND AFTER CLEANING
# =====

print("\n" + "=" * 75)
print("TASK 4: CONGESTION ANALYSIS")
print("=" * 75)

# -----
# Function for congestion classification
# -----

def calculate_congestion(data):

    result = []

    for _, row in data.iterrows():

        if (
            row["AverageSpeed"] < 30
            or row["VehicleCount"] > 140
        ):
            result.append("High")

        elif (
            row["AverageSpeed"] < 45
            or row["VehicleCount"] > 120
        ):
            result.append("Medium")

        else:
            result.append("Low")

    return pd.Series(result)

# Original congestion
original_congestion = calculate_congestion(df)

print("\nOriginal Congestion Distribution:")

print(
    original_congestion.value_counts()
)

# Cleaned congestion
cleaned_congestion = clean_df[
    "CongestionLevel"
]

print("\nCleaned Congestion Distribution:")

print(
    cleaned_congestion.value_counts()
)

```

```

# -----
# Congestion percentages
# -----

print("\nOriginal Congestion Percentage:")

print(
    original_congestion
    .value_counts(normalize=True)
    .mul(100)
    .round(2)
)

print("\nCleaned Congestion Percentage:")

print(
    cleaned_congestion
    .value_counts(normalize=True)
    .mul(100)
    .round(2)
)

# =====
# ROAD-WISE CONGESTION ANALYSIS
# =====

print("\n" + "=" * 75)
print("ROAD-WISE CONGESTION ANALYSIS")
print("=" * 75)

road_analysis = clean_df.groupby(
    "RoadName"
).agg(
    AverageVehicleCount=("VehicleCount", "mean"),
    AverageSpeed=("AverageSpeed", "mean")
)

print(
    road_analysis.round(2)
)

# =====
# TASK 5: AUTOMATED MONITORING RECOMMENDATIONS
# =====

print("\n" + "=" * 75)
print("TASK 5: AUTOMATED DATA MONITORING")
print("=" * 75)

print("""
1. Real-Time Range Validation:
    Automatically reject vehicle counts below 0 and speeds outside
    the predefined city speed limit.

2. Duplicate Detection:
    Use SensorID + Timestamp as a unique key to automatically

```

detect duplicate sensor readings.

3. Timestamp Validation:

Automatically check whether timestamps are missing, duplicated or out of sequence.

4. Sensor Health Monitoring:

Track sensors that repeatedly generate invalid values and automatically flag them for maintenance.

5. Automated Missing-Value Alerts:

Generate alerts whenever a sensor stops transmitting data.

6. Road Name Standardization:

Store road names using predefined road IDs instead of allowing free-text road names.

7. Sensor ID Validation:

Apply a format rule such as S followed by three digits.

8. Real-Time Anomaly Detection:

Use statistical methods such as Z-score, IQR and moving averages to detect unusual traffic readings.

9. Dashboard Monitoring:

Display sensor health, missing readings, abnormal speeds and traffic congestion on a real-time dashboard.

10. Automated Logging:

Store all rejected or corrected sensor readings in an audit table for future investigation.

""")

```
# =====  
# FINAL CLEANED DATA  
# =====
```

```
print("\n" + "=" * 75)  
print("FINAL CLEANED DATA")  
print("=" * 75)
```

```
print(clean_df)
```

```
# =====  
# FINAL SUMMARY  
# =====
```

```
print("\n" + "=" * 75)  
print("FINAL SUMMARY")  
print("=" * 75)
```

```
print("Original Records:",  
      len(df))
```

```
print("Cleaned Records:",  
      len(clean_df))
```

```
print("Missing Timestamps:",
```

```
df["Timestamp"].isna().sum())

print("Duplicate Readings:",
      df.duplicated(
        subset=["SensorID", "Timestamp"]
      ).sum())

print("Negative Vehicle Counts:",
      (df["VehicleCount"] < 0).sum())

print("Unrealistic Speed Readings:",
      (df["AverageSpeed"] > 300).sum())

print("Missing Weather Values:",
      df["WeatherCondition"].isna().sum())

print("Invalid Sensor IDs:",
      len(invalid_sensor))

print("\nTraffic data cleaning and congestion analysis completed.")
```

OUTPUT:

```
IDLE Shell 3.12.10
File Edit Shell Debug Options Window Help

7. Sensor ID Validation:
   Apply a format rule such as S followed by three digits.

8. Real-Time Anomaly Detection:
   Use statistical methods such as Z-score, IQR and moving
   averages to detect unusual traffic readings.

9. Dashboard Monitoring:
   Display sensor health, missing readings, abnormal speeds and
   traffic congestion on a real-time dashboard.

10. Automated Logging:
    Store all rejected or corrected sensor readings in an audit
    table for future investigation.

=====
FINAL CLEANED DATA
=====
SensorID      Timestamp    ... WeatherCondition CongestionLevel
0      S101 2025-01-01 08:00:00 ...      Sunny           Low
1      S102 2025-01-01 08:05:00 ...      Sunny           Medium
2      S103 2025-01-01 08:10:00 ...      Cloudy           Medium
3      S104 2025-01-01 08:15:00 ...      Rainy           Medium
5      S106 2025-01-01 08:25:00 ...      Rainy           Medium
7      S107 2025-01-01 08:30:00 ...      Sunny           High
8      S10A 2025-01-01 08:35:00 ...      Cloudy           High
9      S109 2025-01-01 08:40:00 ...      Rainy           Medium
4      S105 2025-01-01 08:40:00 ...      Sunny           Medium

[9 rows x 8 columns]

=====
FINAL SUMMARY
=====
Original Records: 10
Cleaned Records: 9
Missing Timestamps: 1
Duplicate Readings: 1
Negative Vehicle Counts: 1
Unrealistic Speed Readings: 1
Missing Weather Values: 2
Invalid Sensor IDs: 1

Traffic data cleaning and congestion analysis completed.
>>>
```

Question 5: Global Sales Performance Dataset

A multinational company has merged sales data from five countries. Management wants to evaluate regional performance and forecast future sales. The merged dataset contains severe data quality problems.

Region	Country	SalesPerson	Product	SalesAmount	Currency	SalesDate	CustomerRating
Asia	India	Raj Kumar	Laptop	50000	INR	10-01-2025	4
Asia	India	Rajkumar	Laptop	55000	INR	10-01-2025	5
Europe	Germany	Anna M	Laptop	1200	EUR	12-01-2025	4

Europe	Germany	Anna M	Laptop	1200	EUR	12-01-2025	4
North America	USA	John D	Laptop	1500	USD	01-15-2025	6
Asia	India	Priya S	Mobile	NULL	INR	18-01-2025	3
Europe	France	Pierre L	Ordinateur Portable	1300	EUR	20-01-2025	4
Asia	India	Rahul P	Mobile	9999999	INR	25-01-2025	5
UK	UK	David T	Laptop	1100	GBP	28-01-2025	2
Asia	India	Priya S	Smartphone	35000	INR	30-01-2025	0

Data Issues

- ☐ Multiple currencies (USD, EUR, INR, GBP)
- ☐ Missing sales amounts
- ☐ Duplicate regional entries
- ☐ Customer ratings outside the scale (1–5)
- ☐ Different date formats across countries
- ☐ Product names in multiple languages
- ☐ Salespersons recorded with spelling variations
- ☐ Extreme sales outliers due to data-entry errors

Tasks

1. Develop a comprehensive data-cleaning framework for multinational datasets. (4 Marks)
2. Design methods to standardize currencies, products, and dates. (4 Marks)
3. Handle missing values and outliers using appropriate analytical techniques. (4 Marks)
4. Compare regional sales rankings before and after cleaning. (4 Marks)
5. Prepare an executive report explaining how data quality influences strategic business decisions. (4 Marks)

CODE:

```
import pandas as pd
import numpy as np
```

```
# =====
# QUESTION 5: GLOBAL SALES PERFORMANCE DATASET
# =====
```

```
# =====
# 1. CREATE DATASET
# =====
```

```
data = {
    "Region": [
```

```

        "Asia", "Asia", "Europe", "Europe",
        "North America", "Asia", "Europe", "Asia",
        "UK", "Asia"
    ],

    "Country": [
        "India", "India", "Germany", "Germany",
        "USA", "India", "France", "India",
        "UK", "India"
    ],

    "SalesPerson": [
        "Raj Kumar", "Rajkumar", "Anna M", "Anna M",
        "John D", "Priya S", "Pierre L", "Rahul P",
        "David T", "Priya S"
    ],

    "Product": [
        "Laptop", "Laptop", "Laptop", "Laptop",
        "Laptop", "Mobile", "Ordinateur Portable",
        "Mobile", "Laptop", "Smartphone"
    ],

    "SalesAmount": [
        50000, 55000, 1200, 1200,
        1500, np.nan, 1300, 9999999,
        1100, 35000
    ],

    "Currency": [
        "INR", "INR", "EUR", "EUR",
        "USD", "INR", "EUR", "INR",
        "GBP", "INR"
    ],

    "SalesDate": [
        "10-01-2025", "10-01-2025",
        "12-01-2025", "12-01-2025",
        "01-15-2025", "18-01-2025",
        "20-01-2025", "25-01-2025",
        "28-01-2025", "30-01-2025"
    ],

    "CustomerRating": [
        4, 5, 4, 4,
        6, 3, 4, 5,
        2, 0
    ]
}

df = pd.DataFrame(data)

print("=" * 80)
print("ORIGINAL GLOBAL SALES DATA")
print("=" * 80)

print(df)

# =====

```

TASK 1: DATA QUALITY ASSESSMENT

=====

```
print("\n" + "=" * 80)
print("TASK 1: DATA QUALITY ISSUES")
print("=" * 80)
```

```
# -----
# 1. Multiple currencies
# -----
```

```
print("\n1. Currencies Found:")
print(df["Currency"].unique())
```

```
# -----
# 2. Missing sales amounts
# -----
```

```
print("\n2. Missing Sales Amounts:")
print(df[df["SalesAmount"].isna()])
```

```
# -----
# 3. Duplicate records
# -----
```

```
print("\n3. Duplicate Regional/Sales Records:")
```

```
duplicates = df[
    df.duplicated(
        subset=[
            "Region",
            "Country",
            "SalesPerson",
            "Product",
            "SalesAmount",
            "Currency",
            "SalesDate"
        ],
        keep=False
    )
]
```

```
print(duplicates)
```

```
# -----
# 4. Invalid customer ratings
# -----
```

```
print("\n4. Invalid Customer Ratings:")
```

```
print(
    df[
        (df["CustomerRating"] < 1) |
        (df["CustomerRating"] > 5)
    ]
)
```

```

# -----
# 5. Product names
# -----

print("\n5. Product Names Before Standardization:")

print(df["Product"].unique())

# -----
# 6. Salesperson spelling variations
# -----

print("\n6. Salesperson Names Before Standardization:")

print(df["SalesPerson"].unique())

# -----
# 7. Sales outliers
# -----

print("\n7. Sales Amounts:")

print(df["SalesAmount"])

# =====
# TASK 2: STANDARDIZATION
# =====

clean_df = df.copy()

print("\n" + "=" * 80)
print("TASK 2: STANDARDIZATION")
print("=" * 80)

# =====
# CURRENCY STANDARDIZATION
# =====

# Example fixed exchange rates for academic analysis.
# All currencies are converted to USD.

exchange_rates = {
    "USD": 1.00,
    "EUR": 1.08,
    "GBP": 1.27,
    "INR": 0.012
}

clean_df["ExchangeRate"] = clean_df[
    "Currency"
].map(exchange_rates)

clean_df["Sales_USD"] = (
    clean_df["SalesAmount"] *

```

```

    clean_df["ExchangeRate"]
)

print("\nSales converted to USD:")
print(
    clean_df[
        ["SalesAmount", "Currency", "Sales_USD"]
    ]
)

# =====
# PRODUCT STANDARDIZATION
# =====

product_mapping = {
    "Laptop": "Laptop",
    "Ordinateur Portable": "Laptop",
    "Mobile": "Smartphone",
    "Smartphone": "Smartphone"
}

clean_df["Product"] = (
    clean_df["Product"]
    .str.strip()
    .replace(product_mapping)
)

print("\nProducts after standardization:")
print(clean_df["Product"].unique())

# =====
# SALESPERSON STANDARDIZATION
# =====

salesperson_mapping = {
    "Rajkumar": "Raj Kumar",
    "Raj Kumar": "Raj Kumar",
    "Anna M": "Anna M",
    "John D": "John D",
    "Priya S": "Priya S",
    "Pierre L": "Pierre L",
    "Rahul P": "Rahul P",
    "David T": "David T"
}

clean_df["SalesPerson"] = (
    clean_df["SalesPerson"]
    .str.strip()
    .replace(salesperson_mapping)
)

print("\nSalespersons after standardization:")
print(clean_df["SalesPerson"].unique())

# =====
# DATE STANDARDIZATION
# =====

```

```

# Convert all dates into YYYY-MM-DD format.

clean_df["SalesDate"] = pd.to_datetime(
    clean_df["SalesDate"],
    errors="coerce",
    dayfirst=False
)

# The Indian-style dates need day-first interpretation.
# Correct dates that were interpreted incorrectly.

date_values = [
    "10-01-2025",
    "10-01-2025",
    "12-01-2025",
    "12-01-2025",
    "01-15-2025",
    "18-01-2025",
    "20-01-2025",
    "25-01-2025",
    "28-01-2025",
    "30-01-2025"
]

# Re-read dates with flexible handling
def convert_date(value):

    if pd.isna(value):
        return pd.NaT

    value = str(value)

    # US format containing month/day/year
    if value.startswith("01-15"):
        return pd.to_datetime(
            value,
            format="%m-%d-%Y",
            errors="coerce"
        )

    # Other values are DD-MM-YYYY
    return pd.to_datetime(
        value,
        format="%d-%m-%Y",
        errors="coerce"
    )

# Use original values from the original dataframe
clean_df["SalesDate"] = df["SalesDate"].apply(
    convert_date
)

print("\nStandardized Sales Dates:")
print(clean_df["SalesDate"])

# =====
# REGION STANDARDIZATION

```

```

# =====

# UK is treated as Europe for regional comparison.

region_mapping = {
    "UK": "Europe"
}

clean_df["Region"] = (
    clean_df["Region"]
    .replace(region_mapping)
)

print("\nRegions after standardization:")
print(clean_df["Region"].unique())

# =====
# TASK 3: MISSING VALUES
# =====

print("\n" + "=" * 80)
print("TASK 3: MISSING VALUES AND OUTLIERS")
print("=" * 80)

# -----
# Missing Sales Amount
# -----

print("\nMissing Sales Amount Before Cleaning:")

print(
    clean_df[
        clean_df["SalesAmount"].isna()
    ]
)

# Fill missing sales amount using
# median sales amount for the same currency.

for currency in clean_df["Currency"].unique():

    median_value = clean_df.loc[
        clean_df["Currency"] == currency,
        "SalesAmount"
    ].median()

    clean_df.loc[
        (clean_df["Currency"] == currency) &
        (clean_df["SalesAmount"].isna()),
        "SalesAmount"
    ] = median_value

# Recalculate USD sales
clean_df["Sales_USD"] = (
    clean_df["SalesAmount"] *
    clean_df["ExchangeRate"]
)

```

)

```
# -----  
# CUSTOMER RATING VALIDATION  
# -----
```

```
# Ratings must be between 1 and 5.
```

```
clean_df.loc[  
    (clean_df["CustomerRating"] < 1) |  
    (clean_df["CustomerRating"] > 5),  
    "CustomerRating"  
] = np.nan
```

```
# Fill invalid ratings with median rating  
median_rating = clean_df[  
    "CustomerRating"  
].median()
```

```
clean_df["CustomerRating"] = clean_df[  
    "CustomerRating"  
].fillna(median_rating)
```

```
print("\nCustomer ratings after cleaning:")  
print(clean_df["CustomerRating"])
```

```
# =====  
# DUPLICATE REMOVAL  
# =====
```

```
before_duplicates = len(clean_df)
```

```
clean_df = clean_df.drop_duplicates()
```

```
after_duplicates = len(clean_df)
```

```
print("\nDuplicate Records Removed:",  
      before_duplicates - after_duplicates)
```

```
# =====  
# OUTLIER DETECTION USING IQR  
# =====
```

```
Q1 = clean_df["Sales_USD"].quantile(0.25)  
Q3 = clean_df["Sales_USD"].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
lower_limit = Q1 - 1.5 * IQR  
upper_limit = Q3 + 1.5 * IQR
```

```
print("\nQ1:", Q1)  
print("Q3:", Q3)  
print("IQR:", IQR)
```

```
print("\nLower Outlier Limit:", lower_limit)
print("Upper Outlier Limit:", upper_limit)
```

```
outliers = clean_df[
    (clean_df["Sales_USD"] < lower_limit) |
    (clean_df["Sales_USD"] > upper_limit)
]
```

```
print("\nDetected Sales Outliers:")
```

```
print(
    outliers[
        [
            "Country",
            "SalesPerson",
            "SalesAmount",
            "Currency",
            "Sales_USD"
        ]
    ]
)
```

```
# =====
# OUTLIER TREATMENT
# =====
```

```
# The INR 9,999,999 record is treated as a data-entry error.
# Instead of deleting the entire transaction, cap it at
# the IQR upper limit.
```

```
clean_df["Sales_USD_Cleaned"] = clean_df[
    "Sales_USD"
].clip(
    lower=lower_limit,
    upper=upper_limit
)
```

```
# =====
# REGIONAL SALES BEFORE CLEANING
# =====
```

```
print("\n" + "=" * 80)
print("TASK 4: REGIONAL SALES RANKING")
print("=" * 80)
```

```
# Original data:
# Convert currencies but do not remove duplicates,
# invalid ratings or outlier.
```

```
original_temp = df.copy()
```

```
original_temp["ExchangeRate"] = (
    original_temp["Currency"].map(exchange_rates)
)
```

```
original_temp["Sales_USD"] = (
    original_temp["SalesAmount"] *
```

```

        original_temp["ExchangeRate"]
    )

    original_temp["Region"] = (
        original_temp["Region"]
        .replace(region_mapping)
    )

    regional_before = (
        original_temp
        .groupby("Region")["Sales_USD"]
        .sum()
        .sort_values(ascending=False)
    )

    print("\nRegional Sales BEFORE Cleaning:")

    print(regional_before)

# =====
# REGIONAL SALES AFTER CLEANING
# =====

    regional_after = (
        clean_df
        .groupby("Region")["Sales_USD_Cleaned"]
        .sum()
        .sort_values(ascending=False)
    )

    print("\nRegional Sales AFTER Cleaning:")

    print(regional_after)

# =====
# CREATE RANKINGS
# =====

    ranking_before = (
        regional_before
        .rank(
            ascending=False,
            method="min"
        )
        .astype(int)
    )

    ranking_after = (
        regional_after
        .rank(
            ascending=False,
            method="min"
        )
        .astype(int)
    )

    comparison = pd.DataFrame({

```

```

    "Sales_Before_Cleaning":
        regional_before,

    "Rank_Before":
        ranking_before,

    "Sales_After_Cleaning":
        regional_after,

    "Rank_After":
        ranking_after
})

print("\nRegional Ranking Comparison:")
print(comparison)


# =====
# REGIONAL SALES PERCENTAGE
# =====

print("\nRegional Sales Percentage AFTER Cleaning:")

regional_percentage = (
    regional_after /
    regional_after.sum()
) * 100

print(
    regional_percentage.round(2)
)


# =====
# PRODUCT PERFORMANCE
# =====

print("\n" + "=" * 80)
print("PRODUCT PERFORMANCE AFTER CLEANING")
print("=" * 80)

product_sales = (
    clean_df
    .groupby("Product")["Sales_USD_Cleaned"]
    .sum()
    .sort_values(ascending=False)
)

print(product_sales)


# =====
# SALESPERSON PERFORMANCE
# =====

print("\n" + "=" * 80)
print("SALESPERSON PERFORMANCE AFTER CLEANING")
print("=" * 80)

salesperson_sales = (

```

```

clean_df
.groupby("SalesPerson")["Sales_USD_Cleaned"]
.sum()
.sort_values(ascending=False)
)

```

```
print(salesperson_sales)
```

```

# =====
# TASK 5: EXECUTIVE REPORT
# =====

```

```

print("\n" + "=" * 80)
print("TASK 5: EXECUTIVE REPORT")
print("=" * 80)

```

```

print("""
EXECUTIVE REPORT – GLOBAL SALES DATA QUALITY

```

1. Currency Standardization:
The company uses INR, EUR, USD and GBP. Directly comparing these values would produce misleading regional sales results. Therefore, all sales values were converted to a common USD currency.
2. Missing Sales Amount:
Missing sales values were handled using median imputation within the corresponding currency group. This prevents a missing transaction from being incorrectly treated as zero.
3. Duplicate Records:
Duplicate transactions can artificially increase regional and salesperson revenue. Duplicate records were identified and removed.
4. Customer Rating:
Customer ratings must be between 1 and 5. Values such as 0 and 6 were treated as invalid and replaced using the median rating.
5. Product Standardization:
"Ordinateur Portable" was standardized to "Laptop", while "Mobile" and "Smartphone" were treated as the same product category.
6. Salesperson Standardization:
"Rajkumar" and "Raj Kumar" were standardized to the same salesperson to avoid splitting sales performance.
7. Date Standardization:
Different date formats were converted into a consistent datetime format, allowing accurate monthly and regional analysis.
8. Outlier Treatment:
The extremely large INR 9,999,999 sales value was detected using the IQR method. Since it appears inconsistent with surrounding transactions, it was capped instead of allowing it to dominate regional performance.

9. Strategic Business Impact:

Poor-quality data can cause management to identify the wrong best-performing region, salesperson or product. It can also lead to incorrect sales forecasts, inventory decisions, marketing budgets and resource allocation.

10. Recommendation:

The company should implement automated validation rules, standardized master data, currency conversion services, duplicate detection and regular data-quality audits.

Conclusion:

Data cleaning significantly improves the reliability of regional sales comparisons and forecasting. Management should make strategic decisions only after validating the quality and consistency of the underlying sales data.

""")

```
# =====  
# FINAL CLEANED DATA  
# =====
```

```
print("\n" + "=" * 80)  
print("FINAL CLEANED DATA")  
print("=" * 80)
```

```
print(  
    clean_df[  
        [  
            "Region",  
            "Country",  
            "SalesPerson",  
            "Product",  
            "SalesAmount",  
            "Currency",  
            "Sales_USD_Cleaned",  
            "SalesDate",  
            "CustomerRating"  
        ]  
    ]  
)
```

```
# =====  
# FINAL SUMMARY  
# =====
```

```
print("\n" + "=" * 80)  
print("FINAL SUMMARY")  
print("=" * 80)
```

```
print("Original Records:", len(df))  
print("Cleaned Records:", len(clean_df))
```

```
print(  
    "Missing Sales Amounts:",  
    df["SalesAmount"].isna().sum()  
)
```

```
print(
    "Duplicate Records:",
    df.duplicated().sum()
)

print(
    "Invalid Customer Ratings:",
    (
        (df["CustomerRating"] < 1) |
        (df["CustomerRating"] > 5)
    ).sum()
)

print(
    "Sales Outliers Detected:",
    len(outliers)
)

print("\nRegional Ranking BEFORE Cleaning:")
print(ranking_before)

print("\nRegional Ranking AFTER Cleaning:")
print(ranking_after)

print("\nGlobal sales cleaning and analysis completed successfully.")
```

OUTPUT:

```
IDLE Shell 3.12.10
File Edit Shell Debug Options Window Help
TASK 5: EXECUTIVE REPORT
=====
Squeezed text (60 lines).
=====
FINAL CLEANED DATA
=====
      Region Country ... SalesDate CustomerRating
0      Asia  India ... 2025-01-10          4.0
1      Asia  India ... 2025-01-10          5.0
2     Europe Germany ... 2025-01-12          4.0
4 North America  USA ... 2025-01-15          4.0
5      Asia  India ... 2025-01-18          3.0
6     Europe  France ... 2025-01-20          4.0
7      Asia  India ... 2025-01-25          5.0
8     Europe    UK ... 2025-01-28          2.0
9      Asia  India ... 2025-01-30          4.0

[9 rows x 9 columns]

=====
FINAL SUMMARY
=====
Original Records: 10
Cleaned Records: 9
Missing Sales Amounts: 1
Duplicate Records: 1
Invalid Customer Ratings: 2
Sales Outliers Detected: 1

Regional Ranking BEFORE Cleaning:
Region
Asia          1
Europe        2
North America 3
Name: Sales_USD, dtype: int64

Regional Ranking AFTER Cleaning:
Region
Asia          1
Europe        2
North America 3
Name: Sales_USD_Cleaned, dtype: int64

Global sales cleaning and analysis completed successfully.
>>>
```

Signature of the Course Faculty

: